Deep Learning Foundations

Surrogate models for simulation-driven engineering

Dr. L. Lomazzi
Politecnico di Milano

April 21, 2026

Roadmap

1. From scalar MLP surrogates to richer problems
2. Architectures for fields and meshes: CNN, U-Net, GNN, neural operators
3. Physics-informed neural networks (PINNs)
4. V&V for field and physics-informed surrogates

Prerequisite

Async deck *Basics of Deep Learning for Simulation-Driven Engineering Design*: MLP, training loop, losses, data splits, normalization, overfitting, OOD, uncertainty.

L1 — Recap: the scalar MLP surrogate

From the async deck you already have a working tool:

$$\hat{\boldsymbol{y}} = f_\theta(\boldsymbol{x}), \qquad \theta^\star = \arg\min_\theta \ \frac{1}{N}\sum_{i=1}^{N} \ell\Big(f_\theta(\boldsymbol{x}^{(i)}),\, \boldsymbol{y}^{(i)}\Big)$$

Typical use:

- $\boldsymbol{x}$: design, loading, or material parameters.
- $\hat{\boldsymbol{y}}$: scalar KPIs — maximum deflection, absorbed energy, transmitted impulse.

The starting point

An MLP surrogate replaces an expensive simulation *when the output is a handful of numbers* and inputs live in a low-dimensional parameter space.

L1 — When the scalar MLP is not enough

Three situations where the MLP surrogate hits a wall:

- **Spatial fields.** Output is a displacement or stress map on a grid, not a scalar.
- **Unstructured meshes.** Output lives on nodes of an FE mesh that can change from one simulation to the next.
- **Data-scarce + known PDE.** You have few labeled cases but you do know the governing equations.

Why the MLP struggles

It treats every output component as independent: no spatial structure, no mesh connectivity, no physical link between neighboring points.

L1 — Today's three moves

We extend the toolbox along three directions.

1. **Add spatial structure.** CNNs and U-Nets for field-to-field problems on regular grids.
2. **Go mesh-agnostic.** Graph neural networks (GNNs) for unstructured meshes; neural operators for resolution-independent function-to-function maps.
3. **Inject physics.** PINNs: move the PDE into the loss function.

Takeaway

Each move below is motivated by a concrete limitation of the previous step. The MLP you already know stays as the baseline — every richer architecture still minimizes an empirical loss on data.

L1 — MLP baseline in PyTorch [snippet 1]

```python
import torch
import torch.nn as nn

class MLPSurrogate(nn.Module):
    def __init__(self, d_in, d_out, width=128, depth=4):
        super().__init__()
        layers = [nn.Linear(d_in, width), nn.GELU()]
        for _ in range(depth - 2):
            layers += [nn.Linear(width, width), nn.GELU()]
        layers += [nn.Linear(width, d_out)]
        self.net = nn.Sequential(*layers)
    def forward(self, x):
        return self.net(x)
```

Anchor

Every architecture below keeps the same training loop (forward → loss → backprop → update). Only the model class changes.

L1 — Why MLPs don't scale to fields

Suppose we want stress fields on a 128×128 grid: $m = 16\,384$ outputs.

- A final MLP layer of width 128 already has $W \in \mathbb{R}^{128 \times 16384}$: $\sim 2 \cdot 10^6$ parameters *in one layer*.
- No notion that pixel (i, j) and $(i, j{+}1)$ are neighbors.
- No reuse of a local stress-concentration pattern across the domain.

What we actually need

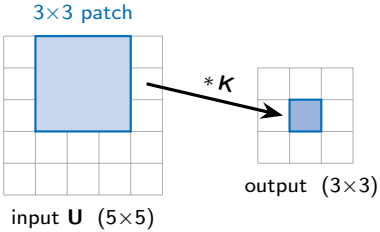
Two inductive biases:

- **Locality**: outputs depend mostly on nearby inputs.
- **Weight sharing**: the same local pattern can appear anywhere in the domain.

L1 – The convolution operation

A convolutional layer slides a small *learnable* kernel $\mathbf{K}$ of size $k \times k$ over the input field:

$$(\mathbf{K} * \mathbf{U})_{ij} \;=\; \sum_{a,b} K_{ab}\, U_{i+a,\, j+b}$$



Three defining properties

- **Local**: output depends on a small neighborhood of the input.
- **Shared weights**: the *same* kernel is applied everywhere $\Rightarrow$ few parameters.
- **Translation-equivariant**: shift the input $\Rightarrow$ the output shifts by the same amount.

L1 – A conv layer in detail

One conv layer is not one kernel – it is a bank of filters.

- C_{out} independent filters, each with weights $\mathbf{K}^{(k)} \in \mathbb{R}^{k \times k \times C_{\text{in}}}$ and a bias.
- Output: C_{out} **feature maps** – each one detects a different pattern.
- Parameter count: $k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$ (e.g. $3 \cdot 3 \cdot 32 \cdot 64 + 64 \approx 18{,}500$).

Two hyperparameters you set.

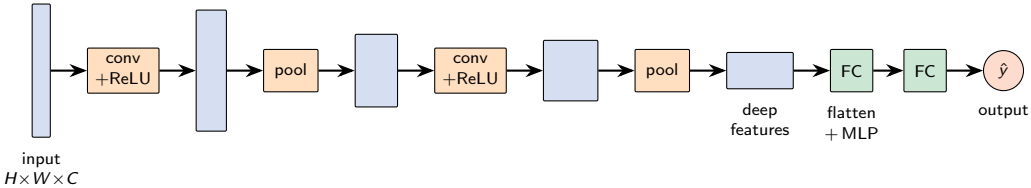
- **Stride** s: step of the slide. $s = 1$ keeps size; $s = 2$ halves it.
- **Padding**: zeros added at the border to control the output size.

The block: conv $\rightarrow$ activation $\rightarrow$ pooling.

- **Activation**: Activation function applied elementwise; the only source of nonlinearity.
- **Pooling** (no learnable weights): partition each feature map into small windows, keep the max (most common) or the mean.
  - Halves spatial resolution, grows the **receptive field** of deeper layers, adds robustness to small shifts.

L1 – A classic CNN architecture

Canonical pattern: stack [Conv → ReLU → Pool] blocks, then a small MLP head.



- **Early layers** – small receptive field ⇒ local patterns (edges, contrasts).
- **Deeper layers** – large receptive field ⇒ structural patterns (load paths, stress concentrations).
- **FC head** – maps the compressed features to the target (scalar QoI, class, small field).

L1 — CNN surrogate: structured-grid fields

A CNN surrogate: input is a loading or geometry map on a regular grid; output is a response field on the same grid.

Typical stack:

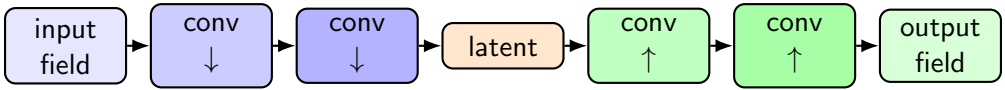
$$\text{input map} \;\rightarrow\; [\text{conv} + \text{activation}]^{L} \;\rightarrow\; \text{output field}$$

Engineering example

Predict a von Mises stress field on a 2D plate given the load pattern and material layout. Once trained, one forward pass replaces one FE solve.

L1 — Encoder–decoder for field-to-field

A symmetric architecture for input field → output field:



- **Encoder**: stacked convs + downsampling compress the input into a low-resolution latent space.
- **Decoder**: stacked convs + upsampling reconstruct the output field.

Bottleneck effect

The latent space forces the model to keep only information relevant for reconstruction — a useful implicit regularizer.

L1 – The plain encoder–decoder

Setup. A *field-to-field* map on the same grid:

$$\mathbf{U} \in \mathbb{R}^{H \times W \times C_{\text{in}}} \longrightarrow \hat{\mathbf{Y}} \in \mathbb{R}^{H \times W \times C_{\text{out}}}$$

e.g. (geometry + loading) $\rightarrow$ stress field.

- **Encoder** E: stack of [conv $\rightarrow$ activation $\rightarrow$ pool] blocks.
  - Spatial size *shrinks*, channels *grow*.
  - Output: compressed latent $\mathbf{z} = E(\mathbf{U})$.
- **Decoder** D: the mirror image, using **upsampling** to grow $\mathbf{z}$ back to $H \times W$.
  - Output: $\hat{\mathbf{Y}} = D(\mathbf{z})$.

L1 – Why a plain encoder–decoder blurs

At the bottleneck, $\mathbf{z}$ has **very small spatial resolution** (e.g. 8×8).

- It *must* discard fine-scale information – there simply isn't room for it.
- The decoder can only *hallucinate* the missing detail from the coarse code.
- Result: smooth predictions, blurred peaks, washed-out fronts.

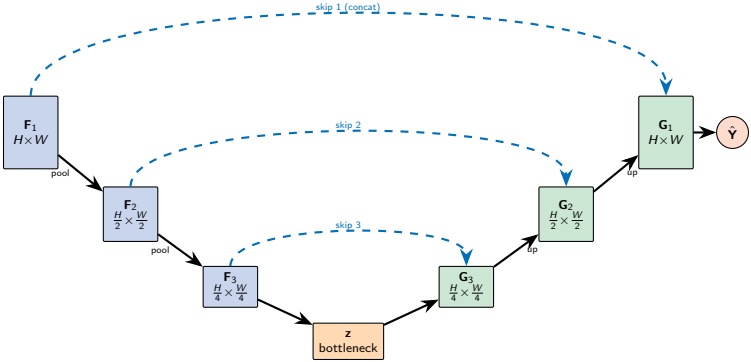
What we actually want

Keep the bottleneck for **global context** (overall load path, geometry) – *and* give the decoder **direct access to the fine-scale features** already computed by the encoder.

$\Rightarrow$ the idea behind the **U-Net**.

L1 – The U-Net: schematic

Idea. Add a **skip connection** at every resolution level: concatenate the encoder's feature map $\mathbf{F}_\ell$ to the decoder's feature map $\mathbf{G}_\ell$ of matching size, *before* the decoder's conv block.



L1 – The U-Net: engineering reading

Variables.

- $\mathbf{F}_\ell$ – encoder feature map at level ℓ (resolution decreases with ℓ).
- $\mathbf{G}_\ell$ – decoder feature map at the same level as $\mathbf{F}_\ell$.
- $\mathbf{z}$ – bottleneck latent; captures global, low-frequency context.
- $\hat{\mathbf{Y}}$ – predicted output field (same resolution as the input).

Why engineers like it

Stress concentrations, shock fronts, contact boundaries are *sharp local features*. The bottleneck $\mathbf{z}$ captures the global load path; the skips let the decoder re-inject the high-frequency detail where a plain encoder–decoder would have blurred it.

L1 — The unstructured-mesh problem

CNNs rely on the grid being **regular**. FE meshes are not:

- Element sizes vary (refined near stress concentrations).
- Topology changes between simulations (different geometries).
- Domains are not rectangular.

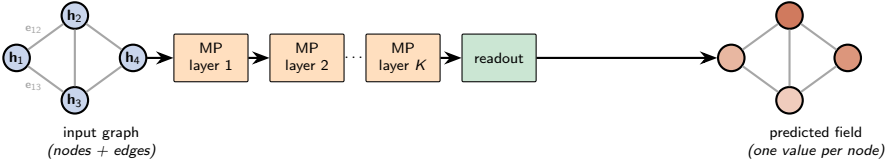
What breaks

There is no single convolution kernel that sweeps an unstructured mesh — "pixel neighbors" are not defined.

Idea: encode the mesh as a graph. Nodes = mesh nodes; edges = element connectivity. Then define a layer that respects that graph structure.

L1 – The GNN at a glance

A GNN takes a graph in and produces predictions per node (or per edge, or per graph).



- **Nodes** v carry features $\mathbf{h}_v$ (coordinates, applied force, material, ...).
- **Edges** (u, v) carry features $\mathbf{e}_{uv}$ (relative position, element type, ...).
- **Message-passing layers** update all node features using local neighborhoods – detailed next.

Analogy with a CNN

CNN: $[\text{conv} \to \text{ReLU} \to \text{pool}]^K \to$ MLP head.
GNN: $[\text{message-passing layer}]^K \to$ readout.
Same recipe – stack local operators, then map to the target.

L1 – One message-passing layer: three steps

Each node v carries a feature vector $\mathbf{h}_v \in \mathbb{R}^d$; each edge a feature $\mathbf{e}_{uv}$ (relative position, element type, ...).

A single layer updates *every* node in parallel, in three steps:

1. **Each neighbor $u \in \mathcal{N}(v)$ sends a message to v:**

$$\mathbf{m}_{u\to v} \;=\; \psi\big(\mathbf{h}_v,\, \mathbf{h}_u,\, \mathbf{e}_{uv}\big).$$

2. **Aggregate** all incoming messages into one vector:

$$\mathbf{m}_v \;=\; \bigoplus_{u\in\mathcal{N}(v)} \mathbf{m}_{u\to v} \qquad \Big(\bigoplus = \text{sum / mean / max}\Big).$$

3. **Update** the node using its old state and the aggregate:

$$\mathbf{h}_v^{\text{new}} \;=\; \phi\big(\mathbf{h}_v,\, \mathbf{m}_v\big).$$

ψ and ϕ are small MLPs, **shared across all edges / nodes** – the same weight-sharing idea as convolution.

L1 – The same thing in one equation

Chaining the three steps gives the standard message-passing update at layer k:

$$\mathbf{h}_v^{(k+1)} = \phi\left(\mathbf{h}_v^{(k)}, \bigoplus_{u \in \mathcal{N}(v)} \psi(\mathbf{h}_v^{(k)}, \mathbf{h}_u^{(k)}, \mathbf{e}_{uv})\right)$$

Variables.

- $\mathbf{h}_v^{(k)}$ – features of node v at layer k.
- $\mathcal{N}(v)$ – neighbors of v (nodes sharing an edge with v).
- $\mathbf{e}_{uv}$ – edge features between u and v.
- ψ, ϕ – message / update MLPs; $\bigoplus$ – permutation-invariant aggregator.

L1 – Why this respects the mesh

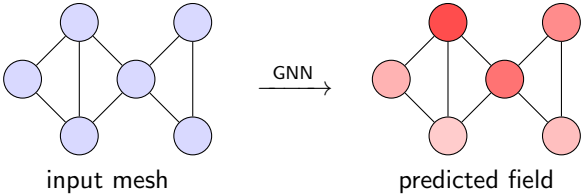
- **Permutation invariance.** $\bigoplus$ does not depend on the order in which neighbors are listed $\Rightarrow$ the result is mesh-topology-agnostic.
- **Weight sharing.** The same ψ, ϕ act on every edge / node – so a GNN trained on one mesh transfers to a different mesh.
- **Receptive field grows by one hop per layer.** After K layers, each node "sees" everything within K edges – the analogue of CNN depth.

The bridge from CNNs

The mesh problem was: no fixed kernel can sweep an unstructured graph. Message passing *is* the kernel – just defined on neighborhoods in the graph rather than pixel windows.

L1 — GNN surrogate for FE meshes

A GNN takes a mesh in, predicts a field on the same mesh out:



Two practical strengths

- Handles different mesh topologies without retraining from scratch.
- Natural fit for dynamic rollouts: predict $\boldsymbol{u}^{(t+1)}$ from $\boldsymbol{u}^{(t)}$ on the same mesh, step by step — the backbone of recent DL solvers for transient simulations.

L1 — A discretization question

Everything so far ties the surrogate to a **specific discretization**:

- CNN $\Rightarrow$ fixed grid resolution.
- GNN $\Rightarrow$ a specific mesh topology (or small variations of it).

Engineering question

Sometimes we want a surrogate for *the mapping itself*, say $f : a(\boldsymbol{x}) \mapsto u(\boldsymbol{x})$, that is independent of how we later discretize the inputs or outputs.

That is the promise of **neural operators**.

L1 — Neural operators: learning function-to-function maps

A neural operator approximates an operator

$$\mathcal{G} : \quad \mathcal{A} \longrightarrow \mathcal{U}, \qquad a \mapsto u,$$

between function spaces, rather than a function on a fixed grid.

- Input: a function a sampled at arbitrary points.
- Output: a function u queryable at arbitrary points.
- Trained once, evaluated at any desired resolution.

Why engineers care

- Train at one resolution, query at another (super-resolution essentially for free).
- Natural fit for *parametric* PDEs where the coefficient field $a(\boldsymbol{x})$ changes between simulations.

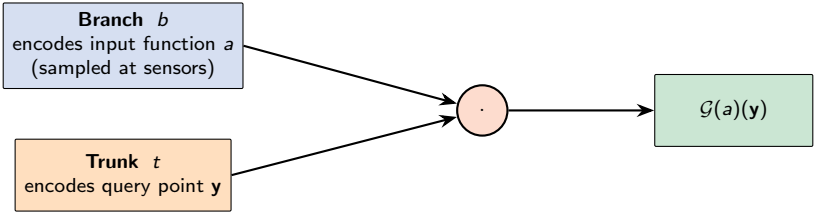
L1 – DeepONet: the idea

Goal. Learn an operator

$$\mathcal{G}: \; a(\mathbf{x}) \longmapsto s(\mathbf{y}), \qquad \text{i.e. } \mathcal{G}(a)(\mathbf{y}) \; = \; s(\mathbf{y}).$$

- $a(\mathbf{x})$ – input function (e.g. a loading field, a conductivity map, an initial condition).
- $\mathbf{y}$ – a query point where we want the output.
- $s(\mathbf{y})$ – value of the output function at $\mathbf{y}$.

DeepONet splits this into two cooperating networks:



L1 – DeepONet: the factorization, in one equation

The branch outputs a vector $\mathbf{b}(a) \in \mathbb{R}^p$, the trunk outputs $\mathbf{t}(\mathbf{y}) \in \mathbb{R}^p$. The operator evaluation is their inner product:

$$\mathcal{G}(a)(\mathbf{y}) \approx \sum_{k=1}^{p} \underbrace{b_k(a)}_{\substack{\text{branch coord} \\ \text{depends on input fn}}} \cdot \underbrace{t_k(\mathbf{y})}_{\substack{\text{trunk coord} \\ \text{depends on query pt}}}$$

- p – shared latent dimension (a hyperparameter, e.g. $p \sim 100$).
- $\mathbf{b}(a)$ is computed **once per input function** a.
- $\mathbf{t}(\mathbf{y})$ is computed **once per query point** $\mathbf{y}$.

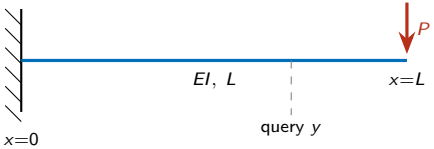
How to think about it – a learned modal expansion

- **Trunk** $\rightarrow$ p learned *mode shapes* $t_k(\mathbf{y})$ – same for every problem.
- **Branch** $\rightarrow$ p *coefficients* $b_k(a)$ – how much of each mode this problem needs.

Both modes and coefficients are discovered from data.

L1 – DeepONet in action: a cantilever with a tip load

Setup. Cantilever beam with length L, bending stiffness EI, free tip load of magnitude P.



Analytical solution (reference). The deflection along the beam is

$$w(x) \;=\; \frac{P}{6EI}\left(3Lx^2 - x^3\right).$$

This is separable: a **coefficient depending on the problem** $P/(6EI)$ times a **shape function of space** $3Lx^2 - x^3$.

L1 – DeepONet in action: a cantilever with a tip load

DeepONet learns this structure from data, with the input function $a(x) \equiv P$ (a constant load magnitude here):

- **Branch** $b(a)$: reads P (and implicitly EI, L) $\rightarrow$ returns the coefficient(s).
- **Trunk** $t(y)$: reads the query location $y \in [0, L]$ $\rightarrow$ returns the shape function(s) evaluated at y.
- **Prediction**: $w(y) \approx b(a) \cdot t(y)$ – same dot-product structure as the analytical solution.

What this example shows

For this simple problem DeepONet essentially *rediscovers* the modal decomposition. For harder problems (distributed load $a(x)$, nonlinear material, complex geometry) the branch and trunk learn modes that no classical basis would suggest – but the mechanism is the same.

L1 — From "learn from data" to "learn from physics and data"

So far, the surrogate learns from $\{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}$ alone.

The PINN idea (Raissi et al., 2019)

If you know the governing PDE, *use it during training*. The network learns to satisfy the available data *and* the equation simultaneously.

This helps most when:

- data are scarce or expensive to generate,
- the PDE is trusted,
- you are solving an inverse problem (e.g., identifying an unknown coefficient).

L1 — PINN ansatz

Represent the solution of a PDE directly as a neural network:

$$u(\boldsymbol{x}, t) \approx u_\theta(\boldsymbol{x}, t)$$

- Inputs: spatial coordinates (and time, if dynamic).
- Output: the field value at that point.
- θ: network weights, learned to satisfy the physics.

Conceptual shift

The network *is* the solution field. There are no elements, no shape functions — the PDE is enforced at sampled "collocation" points inside the domain.

L1 — The PINN loss

Given a PDE $\mathcal{N}[u] = 0$ on domain Ω, boundary condition $u = g$ on $\partial\Omega$, and (optionally) sparse measurements $\{(\boldsymbol{x}_i, u_i)\}$:

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_d} \sum_i \|u_\theta(\boldsymbol{x}_i) - u_i\|^2$$

$$\mathcal{L}_{\text{PDE}}(\theta) = \frac{1}{N_r} \sum_j \|\mathcal{N}[u_\theta](\boldsymbol{x}_j)\|^2$$

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N_b} \sum_k \|u_\theta(\boldsymbol{x}_k) - g(\boldsymbol{x}_k)\|^2$$

$$\boxed{\mathcal{L}(\theta) = \lambda_d \, \mathcal{L}_{\text{data}} + \lambda_r \, \mathcal{L}_{\text{PDE}} + \lambda_b \, \mathcal{L}_{\text{BC}}}$$

Why this works at all

The residual $\mathcal{N}[u_\theta]$ contains derivatives of the network output with respect to its inputs. These are computed *exactly* — not by finite differences — using automatic differentiation.

L1 — PINN residual via autograd [snippet 2]

Example: 1D heat equation $u_t - \alpha\, u_{xx} = 0$.

```
import torch

def pde_residual(net, x, t, alpha):
    x.requires_grad_(True); t.requires_grad_(True)
    u = net(torch.cat([x, t], dim=-1))

    u_t  = torch.autograd.grad(u,  t, torch.ones_like(u),
                               create_graph=True)[0]
    u_x  = torch.autograd.grad(u,  x, torch.ones_like(u),
                               create_graph=True)[0]
    u_xx = torch.autograd.grad(u_x, x, torch.ones_like(u_x),
                               create_graph=True)[0]

    return u_t - alpha * u_xx    # should be 0 for a valid solution
```

Key point

`create_graph=True` keeps the computational graph alive so we can differentiate the residual itself during backprop — no mesh, no finite differences.

L1 — Training a PINN, in practice

Per iteration:

1. Sample collocation points $\{\boldsymbol{x}_j\}$ inside Ω.
2. Sample boundary/initial points when relevant.
3. Forward pass: evaluate u_θ at all sampled points.
4. Compute $\mathcal{L}_{\text{data}}$, $\mathcal{L}_{\text{PDE}}$, $\mathcal{L}_{\text{BC}}$.
5. Backprop the total loss, update θ (Adam is the usual default).

What to monitor

Each loss component *separately*, not only the total. If $\mathcal{L}_{\text{PDE}}$ is ignored by the optimizer, check the weights $\lambda_\bullet$, the collocation sampling strategy, and the smoothness of the architecture.

L1 — PINNs: when they shine, when they struggle

Shine

- Data-scarce forward problems with a clean PDE.
- Inverse problems: identify unknown coefficients from sparse measurements.
- Parametric studies with repeated PDE solves.
- Meshless on arbitrary geometries without remeshing.

Struggle

- Stiff, multi-scale, or shock-dominated problems.
- Complex 3D domains: balancing the loss terms becomes an art.
- Convergence and accuracy guarantees are weaker than FEM.
- Training can be slower than a single FE solve.

Honest take

PINNs are a research-rich tool, not a drop-in replacement for FE solvers. Their clearest industrial wins are in inverse / parameter-identification settings.

L1 — V&V for field and PINN surrogates

Beyond what the async deck covered (scalar V&V, OOD, uncertainty):

- Evaluate error *spatially*: a small mean error can hide a large localized one.
- Test across mesh sizes and grid resolutions, not only held-out samples from the same discretization.
- Check physical consistency: conservation, symmetry, satisfaction of boundary conditions.
- For PINNs: report the PDE residual on a dense test grid, not only on the training collocation points.

Rule

A surrogate that matches integral KPIs but violates physics locally will mislead downstream design decisions.

L1 — Which architecture? A decision table

Input shape	Output shape	Typical choice
low-dim parametric	scalars / short vector	MLP
low-dim parametric	2D/3D field on a grid	CNN decoder / U-Net
2D/3D field on a grid	2D/3D field on a grid	U-Net
unstructured mesh	field on the same mesh	GNN
function on any grid	function on any grid	neural operator (DeepONet)
few data + known PDE	solution field	PINN (next)

Guiding principle

Start from the *shape of your data* and what you know about the physics — not from what is fashionable. Architectures exist because different data carry different structure.

L1 — Summary and bridge to Lecture 2

Today we extended the surrogate toolbox:

- Scalar MLP $\rightarrow$ fields on grids (CNN, U-Net).
- Fields on grids $\rightarrow$ fields on meshes (GNN).
- Fixed discretization $\rightarrow$ resolution-independent maps (neural operators).
- Data-driven $\rightarrow$ physics-informed training (PINNs).

Next lecture: we *flip* the problem.
Instead of predicting behavior from a design, we will *design the structure* to achieve target behavior — with topology optimization accelerated by DL, generative models over the design space, and inverse design.